

Penutupan 4 Gap Checklist Proposal

Historical Shipment Store, Corridor Baseline Learning, Estimated Product Quality, dan Weather-Driven Delay Estimation — dari perencanaan sampai commit.

commit 75f26fc

Branch: fix/hackathon-readiness

Tanggal commit: 23 Agustus 2026

File diubah/ditambahkan: 35 file (+1.828 / -83 baris, di luar PDF)

Terkait: PROPOSAL_ALIGNMENT_CHECKLIST — No. 4, 8, 13, 14

1. Apa yang Berubah

Satu commit ini menutup empat temuan audit kesesuaian proposal yang sebelumnya berstatus / — semuanya sekarang punya infrastruktur nyata di backend (dan sebagian di frontend), bukan lagi klaim tanpa implementasi.

No. Checklist	Temuan Audit	Status Sekarang
4	Historical Shipment — tidak ada data pengiriman aktual, hanya prior sintetis	Infrastruktur dibangun — <code>shipment_records</code> + endpoint <code>outcome</code>
8	Feature Store — tidak ada mekanisme pembelajaran dari data nyata	Corridor Baseline Learning — blend formula ↔ data nyata
13	Route Duration and Delay Estimation — <code>expected_delay_hours</code> selalu 0, tidak menyesuaikan lingkungan	Weather-Driven Delay Estimation — cuaca nyata + check-in per titik
14	Estimated Product Quality — hasil Q10 dihitung tapi tidak diekspos	Diekspos penuh — API + badge di dashboard

2. Ringkasan per Fitur

2.1 Historical Shipment Store

Tabel `shipment_records` mencatat setiap prediksi rute sebagai shipment record. Endpoint `PATCH /shipments/{id}/outcome` memungkinkan pelaporan hasil aktual (delay & status kerusakan sungguhan) — titik data `REAL` pertama yang pernah dimiliki sistem, menggantikan ketiadaan data pengiriman aktual yang ditemukan audit.

2.2 Corridor Baseline Learning

`historical_delay_avg_hours / historical_damage_rate` kini di-blend dengan rata-rata nyata dari laporan outcome begitu satu koridor (moda + bucket jarak) punya cukup sampel — ramp kepercayaan bertahap (3→20 sampel), bukan lompatan tiba-tiba dari formula sintetis ke data nyata.

2.3 Estimated Product Quality

`effective_consumed_hours` hasil simulasi Q10 (sudah dihitung sebelumnya, tapi dibuang) sekarang diekspos sebagai `remaining_shelf_life_hours`, `remaining_shelf_life_pct`, dan `quality_status` (Baik/Menurun/Kritis) — di API dan di dashboard (badge warna + metrik di setiap kartu rute, termasuk perbandingan skenario).

2.4 Weather-Driven Delay Estimation

Rute darat kini mengambil cuaca nyata dari Open-Meteo (dulu selalu "Berawan" tetap). Alur baru: user pilih rute (`POST /select-route`) → check-in di titik-titik perjalanan (`POST /checkpoints` , lewat tombol geolocation browser di dashboard) → saat outcome dilaporkan, sistem menganalisis kecepatan aktual per segmen vs kecepatan nominal rute yang dipilih, mengorelasikannya dengan cuaca saat itu, dan mempelajari delay-akibat-cuaca nyata — menggantikan tabel sintetis murni yang sebelumnya diusulkan.

EVOLUSI DESAIN (DICATAT KARENA BERULANG KALI DISEMPURNAKAN BERSAMA)

Tabel multiplier sintetis murni → data dari "mobile" → analogi Google Maps (GPS kontinu) → disederhanakan jadi check-in per titik (tanpa app, tanpa tracking background) → diikat ke rute spesifik yang dipilih user. Detail lengkap ada di `docs/weather-driven-delay-estimation-plan.pdf` .

2.5 Data Komoditas Digroundkan ke Sumber Resmi

`commodity_database.json` / `commodity_provenance.json` — sebelumnya semua DEMO , sekarang `temp_ideal_min/max_c` dan `shelf_life_hours_at_ideal_temp` disitasi ke FDA/USDA FSIS/UC Davis/IDFA (2 komoditas dikoreksi nilainya: Salmon & Tuna Segar). `delay_tolerance_hours` dan `temp_sensitivity_level` tetap jujur ditandai DEMO karena memang tidak ada padanan di literatur publik.

3. Endpoint API Baru

Endpoint	Fungsi
<code>GET /shipments</code>	Daftar shipment record (filter komoditas/moda/tanggal)
<code>GET /shipments/{id}</code>	Detail satu shipment record
<code>PATCH /shipments/{id}/outcome</code>	Lapor hasil aktual (delay, kerusakan)
<code>POST /shipments/{id}/select-route</code>	Konfirmasi rute mana yang benar-benar dijalankan
<code>POST /shipments/{id}/checkpoints</code>	Check-in titik (lat/lon/waktu) selama perjalanan

Ditambah field baru di response `/predict-route` : `remaining_shelf_life_hours` , `remaining_shelf_life_pct` , `quality_status` , `weather_delay_hours` , `weather_delay_data_quality` .

4. Hasil Pengujian

Suite	Kondisi	Hasil
Seluruh test suite backend	Tanpa Postgres live	52 passed, 7 skipped (butuh DB)
Seluruh test suite backend	Dengan Postgres live	59 passed
Golden demo offline	Tanpa Postgres live	1 passed — tanpa perubahan diperlukan
Frontend <code>npm run build</code>	Type-check penuh	Sukses
Frontend <code>npm run lint</code>	—	Bersih, tanpa isu

Satu bug ditemukan & diperbaiki selama pengujian: cache respons Open-Meteo menyentuh Postgres, tapi awalnya cuma di-`except httpx.HTTPError` — diperluas jadi `except Exception` supaya tetap best-effort saat database mati (konsisten dengan prinsip "DB outage tidak boleh menggagalkan prediksi" di seluruh fitur lain).

5. Dokumentasi Detail per Fitur

[docs/historical-shipment-store.pdf](#)

Skema tabel shipment_records, endpoint outcome, hasil pengujian.

[docs/corridor-baseline-learning.pdf](#)

Mekanisme blend formula ↔ data nyata, tabel corridor_baseline_stats.

[docs/estimated-product-quality.pdf](#)

Field sisa umur simpan & quality_status, perubahan API dan dashboard.

[docs/expected-delay-hours-mechanism.pdf](#)

Riset mekanisme expected_delay_hours sebelum fitur cuaca dibangun.

[docs/weather-driven-delay-estimation-plan.pdf](#)

Rencana lengkap fitur cuaca: taksonomi severity, check-in per titik, pemilihan rute.

6. Daftar File (Ringkas)

[baru] app/routers/shipments.py, app/schemas/shipment_schema.py,
app/services/shipment_service.py

[diubah] app/core/db.py – shipment_records, corridor_baseline_stats, shipment_checkpoints,
weather_delay_stats

[diubah] app/services/historical_baseline.py, enrichment_service.py, temperature_service.py,
weather_service.py

[diubah] app/schemas/route_schema.py, app/data/commodity_database.json,
commodity_provenance.json

[baru] frontend: CheckpointPanel.tsx, QualityBadge.tsx, qualityPalette.ts

[diubah] frontend: page.tsx, CargoTempChart.tsx, ScenarioSimulator.tsx, aiExplain.ts,
types.ts

[baru] tests: test_enrichment_service.py, test_historical_baseline.py,
test_weather_service.py, test_shipment_store_e2e.py, test_shipment_checkpoints_e2e.py

[diubah] tests: test_api.py, test_commodity_service.py, test_predict_route_e2e.py,
test_scenario_e2e.py